MOBILE CORE HOME LAB · MODULE 1

# 5G Standalone, hands-on: what was built and why

A plain-language walkthrough of Module 1. It starts simple and then goes into the details of the captures in this repository: what a 5G core is, how a phone registers and gets internet access, what was broken on purpose, and how to find the same faults in a live network.

| | |
|---|---|
| **Core** | Open5GS 2.8 (all 5G SA network functions on one VM) |
| **Radio and phone** | UERANSIM 3.3.0: software gNB and software UE, nothing transmitted over the air |
| **Network (PLMN)** | MCC 250, MNC 99. Test subscriber IMSI 250990000000001 |
| **Host** | Windows 11 Pro → Hyper-V → Ubuntu 26.04 VM, 4 vCPU, 4 GB RAM |
| **Repository** | github.com/smklog/mobile-core-lab, folder 01-5gsa-core |
| **Date** | October 26, 2026 |

**Contents**


1. Why this lab exists
2. Glossary: every acronym spelled out
3. How a 5G SA network is built
4. How the lab is built
5. What was done, step by step
6. Procedure 1: Registration and 5G-AKA
7. Procedure 2: PDU Session (internet access)
8. Procedure 3: User data on N3 (GTP-U)
9. Procedure 4: Deregistration
10. Procedure 5: Idle mode and Service Request
11. Six failure scenarios F1–F6
12. Reading the captures in Wireshark
13. Key takeaways

# 1. Why this lab exists

The goal of the whole lab is to show **hands-on** knowledge of the mobile packet core. Not "read the specs", but brought up a network, looked at every packet, and can find faults.

The repository holds three kinds of evidence:

- **A working setup**: scripts and instructions to reproduce it.
- **Packet captures (pcap)** with a step-by-step analysis: which message, from whom to whom, and what matters in it.
- **Break/fix scenarios**: something is broken on purpose, the symptoms are recorded, the cause is found and fixed. This is the classic troubleshooting question: "a subscriber has no data, where do you look first?"

Module 1 is the foundation: a 5G core in Standalone mode. Later modules (LTE/EPC and Diameter, IMS and VoLTE, MPLS transport, SS7) build on the same understanding.

> **Why a local VM and a simulated radio.** The core needs SCTP, TUN interfaces and root privileges, so the lab runs in a VM on a home PC. The radio is entirely software (UERANSIM): nothing is transmitted over the air and no spectrum license is involved.

# 2. Glossary

Full name first, then what it means in plain words. Come back to this table whenever an unfamiliar term shows up later.

## General terms

| Acronym | Full name | In plain words |
|---|---|---|
| **3GPP** | 3rd Generation Partnership Project | The body that writes mobile standards (TS 23.501, 24.501, etc.) |
| **5G SA** | 5G Standalone | "Real" 5G: 5G radio and 5G core, no LTE involved |
| **5G NSA** | 5G Non-Standalone | 5G radio on top of the older LTE core (EPC). A transition option |
| **5GC** | 5G Core | The 5G core network: a set of network functions (AMF, SMF, UPF...) |
| **NF** | Network Function | One core program with one role |
| **UE** | User Equipment | The subscriber device: phone, modem, sensor |
| **gNB** | next generation NodeB | The 5G base station (cell site). LTE used eNB |
| **NG-RAN** | Next Generation Radio Access Network | The 5G radio network: all gNBs together |
| **Control plane** | | Signaling: "who are you", "give me a session", "go idle" |
| **User plane** | | The subscriber's actual data: web, video, our ping |
| **EPC** | Evolved Packet Core | The LTE (4G) core. Module 2. Open5GS runs it too |

## 5G core network functions

| Acronym | Full name | What it does |
|---|---|---|
| **AMF** | Access and Mobility Management Function | The "front desk". Admits the phone, has it authenticated (via AUSF), tracks where it is and its state. Counterpart of the LTE MME |
| **SMF** | Session Management Function | Creates and deletes data sessions, assigns the IP address, controls the UPF |
| **UPF** | User Plane Function | The data "router": terminates the tunnel from the gNB and sends traffic to the internet |
| **NRF** | Network Repository Function | The "directory": NFs register there and discover each other |
| **AUSF** | Authentication Server Function | Runs subscriber authentication together with UDM |
| **UDM** | Unified Data Management | Subscriber data logic: keys, subscription, serving AMF. Counterpart of the HSS |
| **UDR** | Unified Data Repository | The data store. Here it reads from MongoDB |
| **PCF** | Policy Control Function | Policy: bit rates, QoS, access rules. Counterpart of the PCRF |
| **NSSF** | Network Slice Selection Function | Helps select the network slice |
| **BSF** | Binding Support Function | Remembers which PCF serves which session |

| | | |
|---|---|---|
| **SCP** | Service Communication Proxy | Proxy between NFs. All requests here go through it, so every request appears twice in the captures |
| **SIDF** | Subscription Identifier De-concealing Function | The part of UDM that turns the concealed SUCI back into the SUPI |

## Interfaces and protocols

| Name | Full name | Where it is used |
|---|---|---|
| **Uu** | radio interface | UE ↔ gNB, the air interface. Simulated here |
| **N1** | reference point N1 | UE ↔ AMF, logical. NAS messages, physically carried through the gNB |
| **N2** | reference point N2 | gNB ↔ AMF. NGAP over SCTP, port 38412 |
| **N3** | reference point N3 | gNB ↔ UPF. User data in GTP-U tunnels, UDP 2152 |
| **N4** | reference point N4 | SMF ↔ UPF. PFCP, UDP 8805 |
| **N6** | reference point N6 | UPF ↔ internet (Data Network) |
| **SBI** | Service Based Interface | How NFs talk to each other: HTTP/2 + JSON, TCP 7777 here |
| **NAS** | Non-Access Stratum | Phone ↔ core messages that the gNB does not read, only relays |
| **5GMM** | 5G Mobility Management | The NAS part for registration and access. Its causes: #20, #62, #91… |
| **5GSM** | 5G Session Management | The NAS part for data sessions. Its causes: #27, #38… |
| **NGAP** | NG Application Protocol | The N2 protocol (gNB ↔ AMF) |
| **SCTP** | Stream Control Transmission Protocol | Transport for NGAP, like TCP but built for signaling |
| **PFCP** | Packet Forwarding Control Protocol | How the SMF programs the UPF: "send packets like these there" |
| **GTP-U** | GPRS Tunnelling Protocol, User plane | The tunnel for subscriber data between gNB and UPF |
| **HTTP/2** | Hypertext Transfer Protocol version 2 | Transport for the SBI |
| **RRC** | Radio Resource Control | Radio signaling UE ↔ gNB |

## Identifiers

| Acronym | Full name | In plain words (value in this lab) |
|---|---|---|
| **PLMN** | Public Land Mobile Network | The operator network ID = MCC + MNC (250/99 here) |
| **MCC** | Mobile Country Code | Country code (250) |
| **MNC** | Mobile Network Code | Operator code within the country (99) |
| **IMSI** | International Mobile Subscriber Identity | The permanent SIM identity: MCC+MNC+MSIN (250990000000001) |
| **MSIN** | Mobile Subscription Identification Number | The part of the IMSI after MCC and MNC (0000000001) |

| | | |
|---|---|---|
| **SUPI** | Subscription Permanent Identifier | The permanent 5G identity. Usually the IMSI |
| **SUCI** | Subscription Concealed Identifier | The encrypted SUPI, so the IMSI never goes over the air in clear. This lab uses the "null" scheme, so the MSIN is visible |
| **5G-GUTI** | 5G Globally Unique Temporary Identifier | Temporary ID the AMF gives the phone instead of the SUPI |
| **5G-TMSI / 5G-S-TMSI** | 5G Temporary Mobile Subscriber Identity / Shortened | The short part of the GUTI. Used to wake up (Service Request) |
| **TAC / TAI** | Tracking Area Code / Identity | The area where the UE is (TAC 1 here) |
| **DNN** | Data Network Name | Name of the data network to connect to ("internet"). Called APN in LTE |
| **APN** | Access Point Name | The 4G equivalent of DNN |
| **S-NSSAI** | Single Network Slice Selection Assistance Information | One slice ID = SST (+ optional SD) |
| **SST / SD** | Slice/Service Type / Slice Differentiator | Slice type (1 = regular mobile broadband) and a sub-ID |
| **NSSAI** | Network Slice Selection Assistance Information | A list of slices. Requested NSSAI from the UE, Allowed NSSAI from the network |
| **PDU Session** | Protocol Data Unit Session | The phone's "internet connection": IP address + tunnel + rules |
| **PSI** | PDU Session Identity | Session number on the phone (1 here) |
| **TEID** | Tunnel Endpoint Identifier | GTP-U tunnel ID. Each side assigns its own |
| **F-TEID** | Fully qualified TEID | IP address + TEID together |

## Security

| Acronym | Full name | In plain words |
|---|---|---|
| **K** | Subscriber Key | The secret key: stored in the SIM and in the operator database, never sent anywhere |
| **OPc** | Operator Variant Algorithm Configuration Field (derived) | The second secret, the "operator signature". Must match in SIM and database |
| **SQN** | Sequence Number | A counter that prevents replay of old challenges |
| **5G-AKA** | 5G Authentication and Key Agreement | Mutual phone ↔ network authentication and key derivation |
| **RAND** | Random challenge | The random "puzzle" from the network |
| **AUTN** | Authentication Token | Proof that the challenge came from the real network (carries SQN and MAC) |
| **MAC** | Message Authentication Code | The check value inside AUTN |
| **RES* / XRES*** | Response / Expected Response | The phone's answer and the answer the network expects. A match means a genuine subscriber |
| **AUTS** | Re-synchronisation Token | "My SQN is different, please resync" |

| | | |
|---|---|---|
| **NEA / NIA** | NR Encryption / Integrity Algorithm | Ciphering and integrity algorithms. EA0 = no ciphering, IA2 = AES-based |
| **ngKSI** | Key Set Identifier | Key set number. Tells UE and AMF the keys are already in place |

## QoS, rules, states

| Acronym | Full name | In plain words |
|---|---|---|
| **QoS** | Quality of Service | Bit rate, priority, latency |
| **QFI** | QoS Flow Identifier | QoS flow number inside a session (1 here) |
| **5QI** | 5G QoS Identifier | QoS class (9 = default best-effort internet) |
| **AMBR** | Aggregate Maximum Bit Rate | Bit rate cap (1 Gbps here) |
| **PDR** | Packet Detection Rule | UPF rule: "which packets to match" |
| **FAR** | Forwarding Action Rule | UPF rule: "what to do with them" |
| **FORW / BUFF / NOCP** | Forward / Buffer / Notify the CP function | FAR actions: forward / buffer / notify the SMF |
| **RM-REGISTERED** | Registration Management | The phone is registered |
| **CM-CONNECTED / CM-IDLE** | Connection Management | Signaling connection to the core exists / the phone is idle |

## Linux tools

| Name | Full name | Why it matters here |
|---|---|---|
| **VM, Hyper-V** | Virtual Machine, Microsoft hypervisor | Linux on a Windows host |
| **TUN** | network tunnel interface | A virtual NIC inside a program: ogstun on the UPF, uesimtun0 on the UE |
| **netns** | network namespace | A separate network stack inside Linux with its own routes |
| **NAT** | Network Address Translation | Rewrites 10.45.x.x to the VM address to reach the internet |
| **iptables** | Linux packet filter | NAT and forwarding rules |
| **pcap** | packet capture | A file of captured packets (tcpdump writes it, Wireshark reads it) |

## 3. How a 5G SA network is built

Think of the network as a company. The **gNB** is the entrance: it talks to the phone over the radio. The **AMF** is the front desk: it checks credentials and knows who is where. The **SMF** is the account manager who sets up the "internet connection". The **UPF** moves the actual data. **UDM/UDR** is HR with the subscriber files. The **NRF** is the company directory.

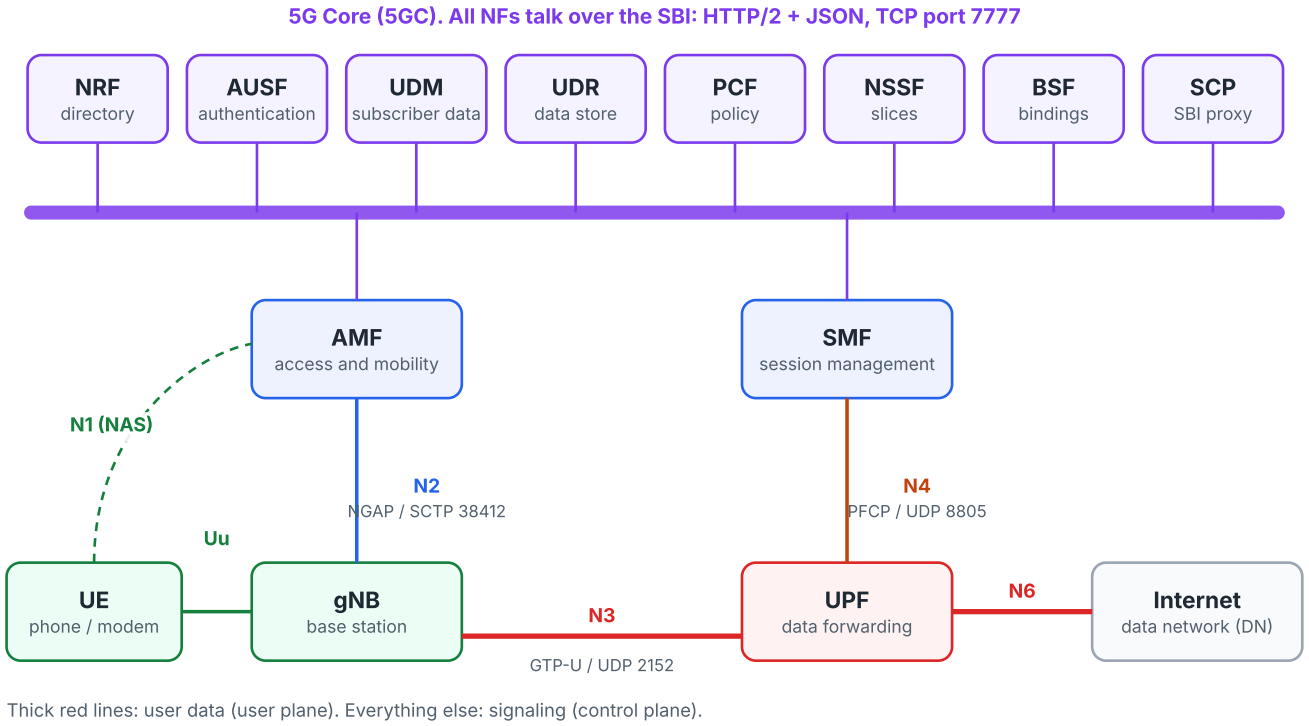



Figure 1. 5G SA architecture and interfaces. Port numbers are the ones used in this lab.

Two key ideas:

1. **Control is separate from data.** Signaling (who, where, which rights) goes through the AMF/SMF. Subscriber data only travels UE → gNB → UPF → internet. The AMF and SMF never see it; the SMF only instructs the UPF over N4.
2. **Service-based architecture (SBI).** NFs are web services. They send each other HTTP/2 requests such as `POST /nausf-auth/v1/ue-authentications`. The path always shows which NF is called: `nausf` = AUSF, `nudm` = UDM, `nsmf` = SMF, `npcf` = PCF, `nnrf` = NRF.

# 4. How the lab is built

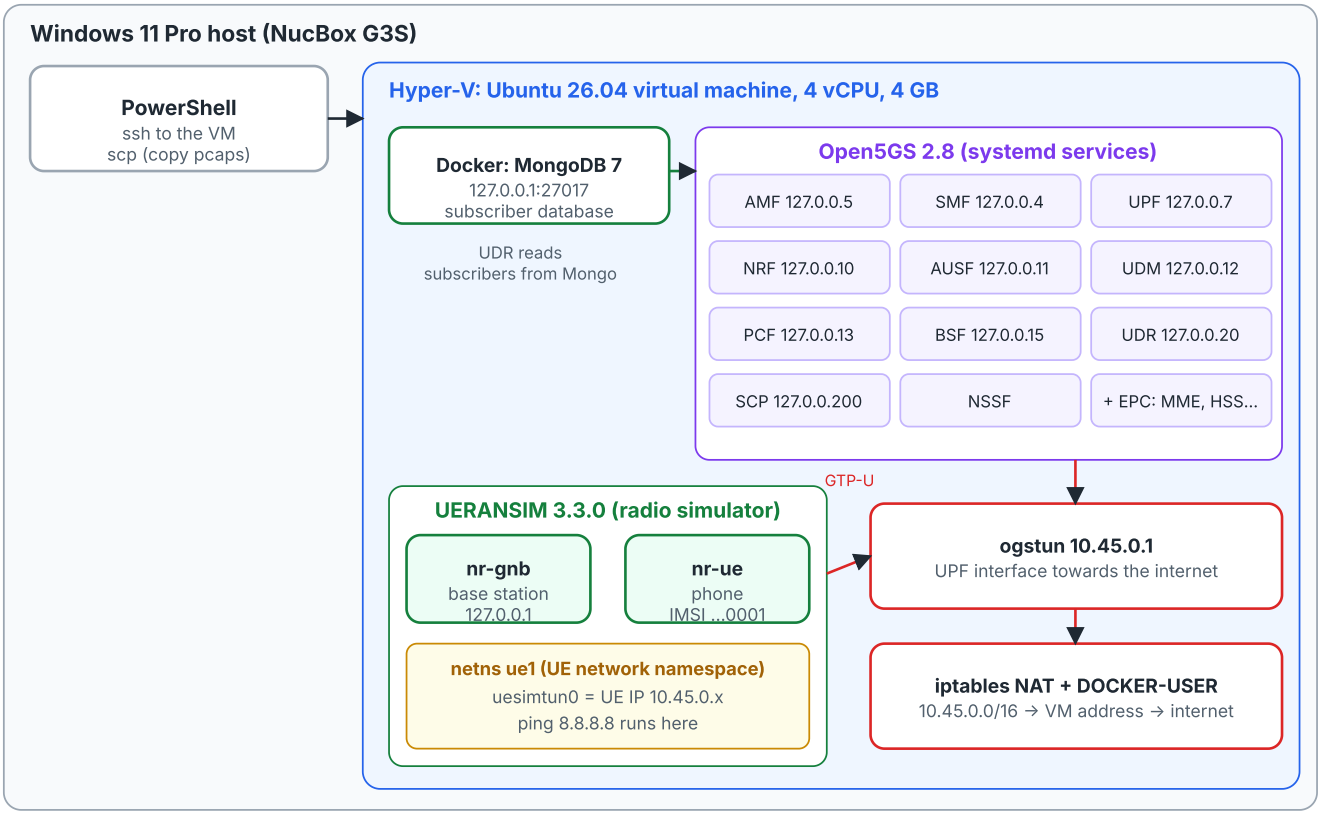



Figure 2. Everything runs on one VM. The NFs use separate 127.0.0.x addresses, so all core traffic crosses the loopback interface and is captured with `tcpdump -i lo`.

| Address | Node | Address | Node |
|---|---|---|---|
| 127.0.0.1 | gNB (UERANSIM), also NF client side | 127.0.0.11 | AUSF |
| 127.0.0.5 | AMF (NGAP 38412, SBI 7777) | 127.0.0.12 | UDM |
| 127.0.0.4 | SMF (SBI, PFCP 8805) | 127.0.0.13 | PCF |
| 127.0.0.7 | UPF (PFCP 8805, GTP-U 2152) | 127.0.0.15 | BSF |
| 127.0.0.10 | NRF | 127.0.0.20 | UDR |
| 127.0.0.200 | SCP | 10.45.0.0/16 | UE IP address pool |

## 5. What was done, step by step

1. **VM.** Ubuntu 26.04 on Hyper-V (4 vCPU, 4 GB). Checked that the `sctp` module and `/dev/net/tun` are available: the core and UERANSIM need both.
2. **Install.** Open5GS 2.8 from the PPA. MongoDB 7 in Docker, because there is no official package for Ubuntu 26.04 and a container does not care. UERANSIM built from source (`make`).
3. **Own network 250/99.** MCC/MNC changed to 250/99 in every Open5GS config (`/etc/open5gs/*.yaml`), plus `lab-gnb.yaml` and `lab-ue.yaml` for UERANSIM. Script: `set-plmn.sh`.
4. **Subscriber.** IMSI 250990000000001 added with test K and OPc (`add-subscriber.sh`, which wraps `open5gs-dbctl`). Test keys only, the same ones UERANSIM ships with.
5. **First run.** The gNB connected to the AMF (NG Setup), the UE registered and received a 10.45.0.x address.
6. **Three problems solved** (typical when everything shares one host):
   - **UERANSIM could not create its TUN:** "Could not open /etc/iproute2/rt_tables". On Ubuntu 26.04 the file lives in `/usr/share/iproute2`; copied it.
   - **No internet from the UE.** Docker sets `iptables -P FORWARD DROP` on install, which blocks forwarding. Allowed ogstun in the DOCKER-USER chain and added NAT for 10.45.0.0/16. `host-setup.sh` does this; run it after every reboot.
   - **"Time to live exceeded" on ping.** Packets looped (Figure 3). Fixed by moving the UE interface `uesimtun0` into its own network namespace `ue1` (`ue-netns.sh`).
7. **Captures.** Registration, PDU session, deregistration, N3 traffic and Service Request were captured and analyzed in `callflow.md`.
8. **Failures F1–F6.** Six break → observe → diagnose → fix scenarios in `failures.md`.

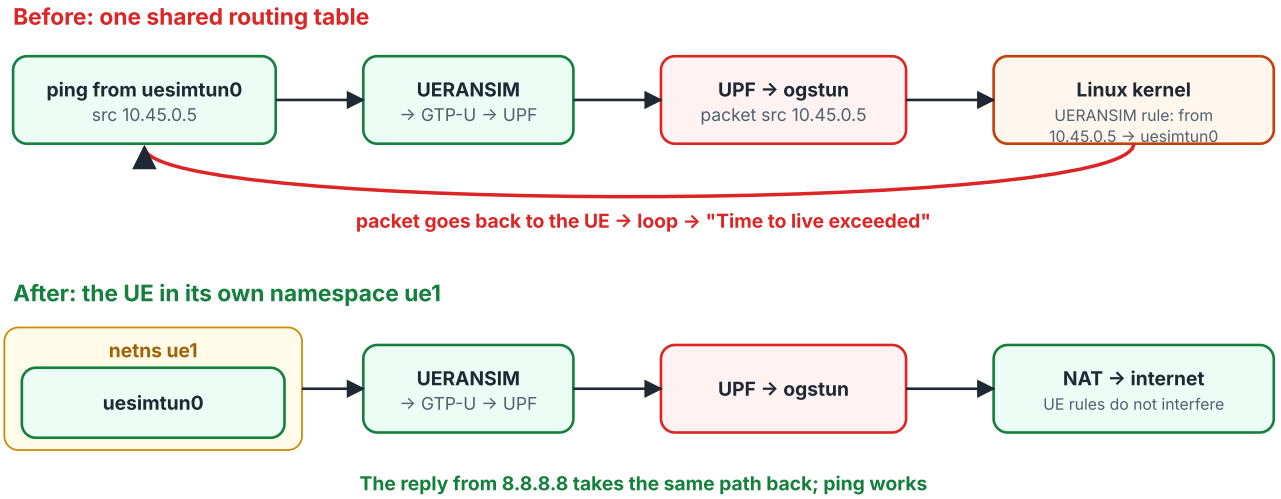



Figure 3. Why ping failed while the UE and the UPF shared one routing table, and how a namespace fixed it.

# 6. Procedure 1: Registration and 5G-AKA

**Why:** before the phone may do anything, the network has to learn who it is, verify that the SIM is genuine and turn on message protection. The phone, in turn, verifies that the network is genuine. Authentication is mutual.

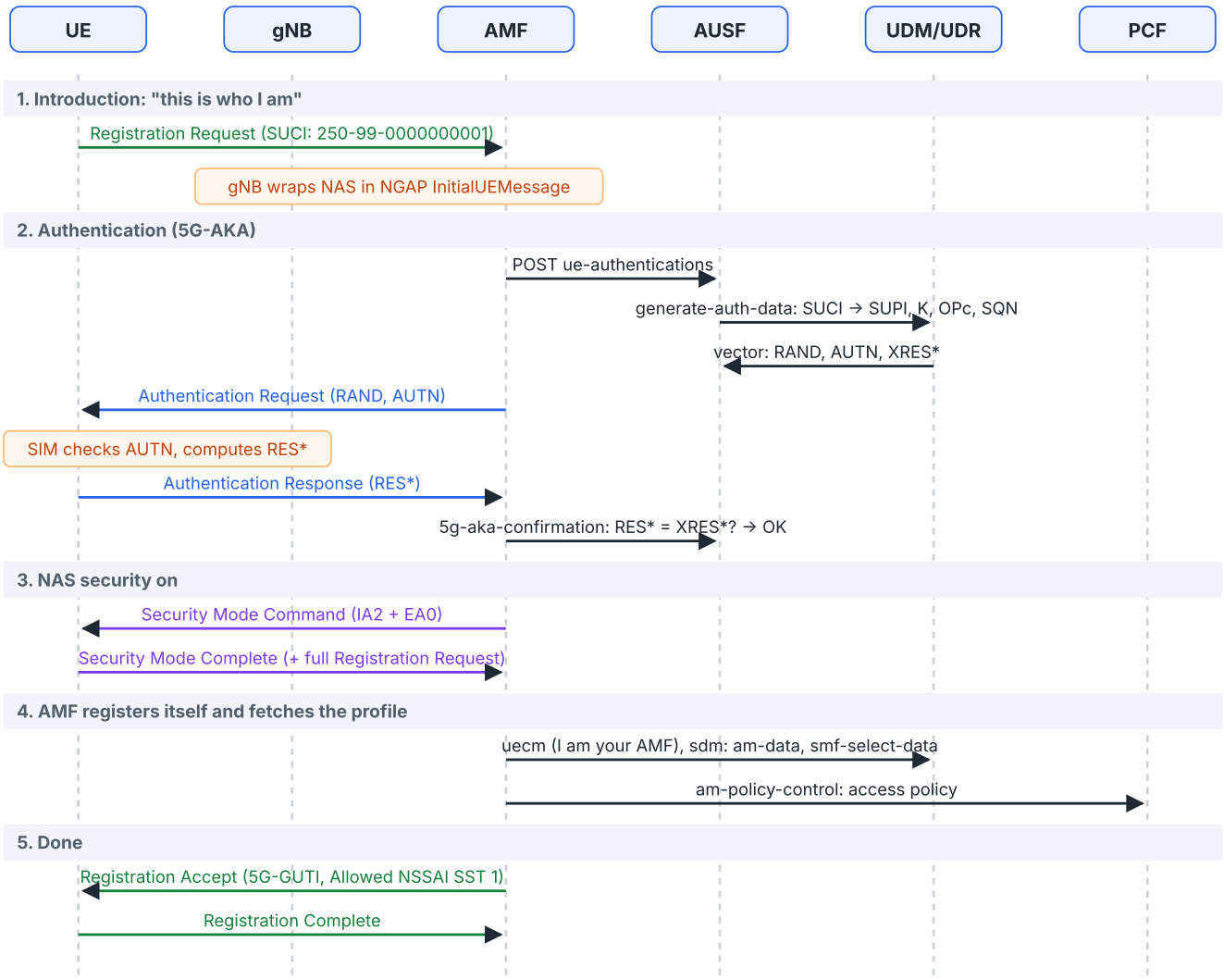



Figure 4. Registration (simplified). Frames 1205–1509 in m1-registration.pcap.

## 5G-AKA without the math

The SIM and the operator database hold the same secrets **K** and **OPc**, plus a counter **SQN**. None of them ever goes over the network.

1. UDM picks a random **RAND** and from K, OPc and SQN computes **AUTN** (proof "I am the real network") and **XRES*** (the answer it expects from the phone).
2. Only RAND and AUTN are sent to the phone.
3. The SIM checks AUTN with its own K and OPc. If the MAC does not match, the network is fake or the keys differ: the phone answers *Authentication Failure #20 MAC failure*. If only the SQN is out of range, the answer is *#21 Synch failure* with AUTS, and the network resynchronizes SQN.

4. If all is well, the SIM computes **RES*** and sends it. AUSF compares RES* with XRES*. A match: genuine subscriber.
5. Both sides derive keys from the same secrets. The AMF then turns protection on with **Security Mode Command**: here integrity **128-5G-IA2** and ciphering **5G-EA0** (no ciphering, which is why NAS is readable in Wireshark).

> **Seen on the first run.** A Synch failure came first: SQN in the database was 0 while the UE expected another value. UDM resynchronized SQN from AUTS and the second attempt passed. That is the mechanism working, not a fault.

## Key frames (m1-registration.pcap)

| Frame | Message | What to look at |
|---|---|---|
| 1205 | InitialUEMessage + Registration Request | SUCI suci-0-250-99-0000-0-0-0000000001, null protection scheme |
| 1206–1313 | old session cleanup | the UE was restarted without deregistering; the AMF cleans up the old context |
| 1315, 1333 | AUSF, UDM generate-auth-data | 5G-AKA starts; UDM de-conceals SUCI to SUPI |
| 1364 / 1365 | Authentication Request / Response | RAND, AUTN / RES* |
| 1398 / 1399 | Security Mode Command / Complete | IA2 + EA0; the Complete carries the full Registration Request |
| 1504 | InitialContextSetupRequest + Registration Accept | 5G-GUTI (AMF Region 2, Set 1, 5G-TMSI 0xc0000189), Allowed NSSAI SST 1 |
| 1509 | Registration Complete | the PDU session request rides in the same packet |

# 7. Procedure 2: PDU Session (internet access)

**Why:** registration only gives the right to be on the network. To reach the internet the phone needs a **PDU session**: an IP address, a tunnel from the gNB to the UPF, and QoS rules. The phone requests it from the SMF (through the AMF) with DNN "internet" and slice SST 1.

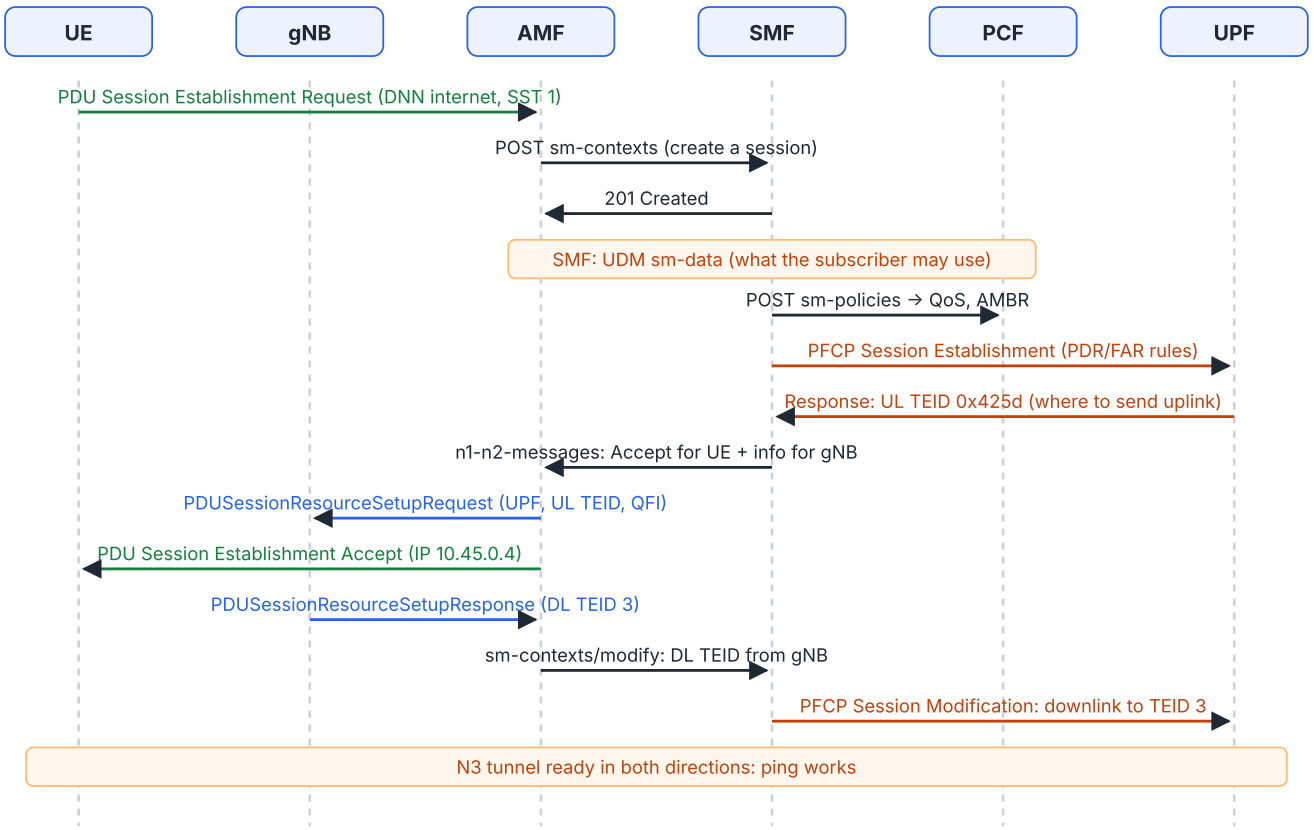



Figure 5. PDU session establishment. Frames 1509–1652 in m1-registration.pcap.

The key point: **the tunnel is built from two halves**. The UPF assigns the uplink TEID (0x425d), the gNB assigns the downlink TEID (3). The SMF joins them: first it installs rules on the UPF (PFCP Session Establishment), then, once it learns the gNB downlink TEID, it updates them (PFCP Session Modification).

| Frame | Message | What to look at |
|---|---|---|
| 1511 | POST /nsmf-pdusession/v1/sm-contexts | the AMF selected an SMF and asks it to create the session |
| 1556 | POST /npcf-smpolicycontrol/v1/sm-policies | session policy: 5QI 9, AMBR |
| 1597 / 1598 | PFCP Session Establishment | PDR/FAR rules on the UPF, uplink F-TEID |
| 1617 | PDUSessionResourceSetupRequest + Accept | UE IP 10.45.0.4, DNS 8.8.8.8, AMBR 1 Gbps |
| 1626 | PDUSessionResourceSetupResponse | downlink TEID 3 from the gNB |
| 1627–1634 | modify + PFCP Session Modification | the UPF learns where to send downlink |

# 8. Procedure 3: User data on N3

**Why it matters:** "how does a user packet cross the core" is a common question. The answer is in `m1-n3-gtpu.pcap`.

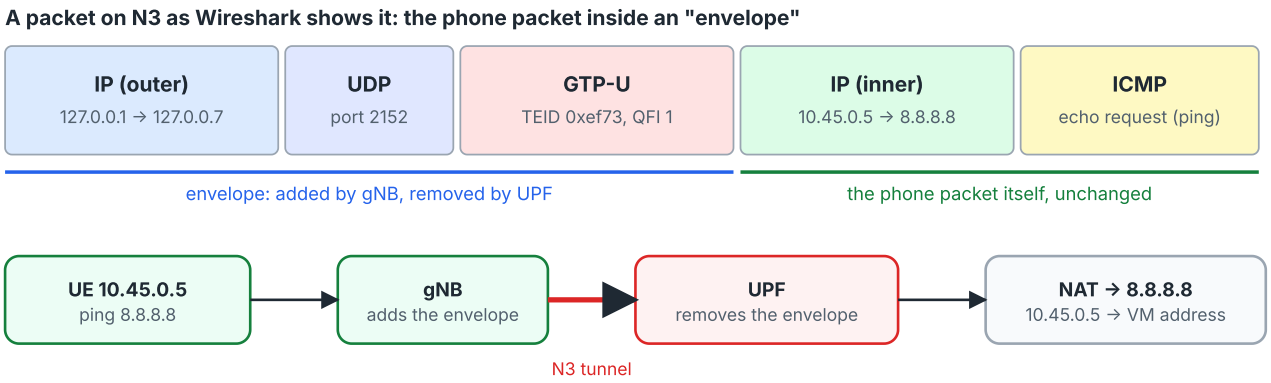



Figure 6. A ping packet inside GTP-U. The gNB adds the outer IP, UDP and the GTP-U header with the TEID; the UPF strips them and sends the inner packet to the internet.

- Uplink (gNB → UPF) TEID 0x0000ef73, assigned by the UPF. Downlink (UPF → gNB) TEID 0x00000004, assigned by the gNB. **Rule: the receiver assigns the TEID it wants to receive on.**
- The GTP-U header carries the PDU Session Container extension with **QFI 1**, so the UPF and gNB know which QoS flow the packet belongs to.
- Ping RTT about 8 ms, all 5 replies received.

## 9. Procedure 4: Deregistration

Triggered with `nr-cli imsi-250990000000001 -e "deregister normal"`. Order: the UE sends Deregistration Request → the AMF asks the SMF to release the session → the SMF deletes the UPF rules (PFCP Session Deletion) → PCF, BSF and UDM state is cleaned up → the AMF sends Deregistration Accept → the gNB releases the connection (UEContextRelease). Frames 2375–2490.

The point: on a clean deregistration **the network removes everything itself**: session, tunnel, policies, the UDM registration. If the phone disappears without deregistering (as when the UE process was restarted), the AMF cleans up at the next registration, as seen in frames 1206–1313.

# 10. Procedure 5: Idle mode and Service Request

**Why:** a phone cannot keep the radio connection up all the time; that drains the battery. With no data, the network releases it to **CM-IDLE**. It stays registered and keeps its IP address and session, but the tunnel to the gNB is torn down. When the phone has data again, it sends a **Service Request** and wakes up quickly.

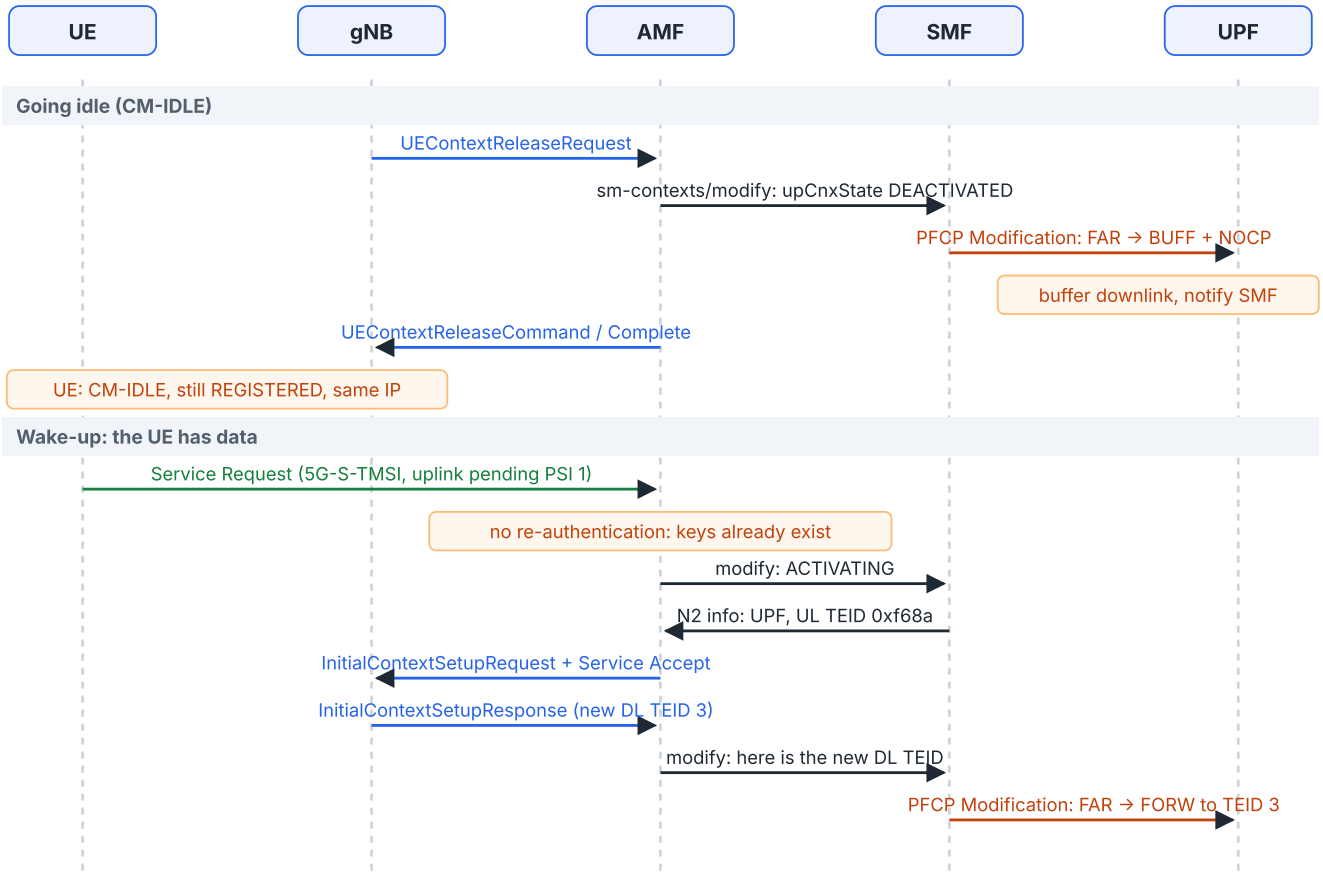



Figure 7. Going idle and waking up. m1-service-request.pcap, 40 frames.

- The UE was released with the gNB command `ue-release 5`. The AMF told the SMF the connection is inactive. The SMF switched the UPF rule to **BUFF + NOCP**: buffer downlink data and notify the SMF. Downlink data at that point would make the network page the phone (not tested here).
- The Service Request carries the 5G-S-TMSI (0xc0000075), not the SUCI. **No re-authentication**: the keys are already in place (ngKSI 3). Service Request to Service Accept took 7 ms.
- The tunnel is rebuilt: the UPF uplink TEID stays the same (0xf68a), but the gNB assigns a **new** downlink TEID (2 → 3), because it dropped the UE context on release. The SMF switches the rule back to **FORW** towards the new TEID.
- One open point is recorded in the analysis: the UE flagged pending uplink data at 22:20:14, but the ping started at 22:20:20. The packet that woke it up was not identified.

# 11. Six failure scenarios F1–F6

Each scenario breaks one link. The figure shows where. The skill being practiced: **from the symptom and the cause code, tell who rejected and why**.

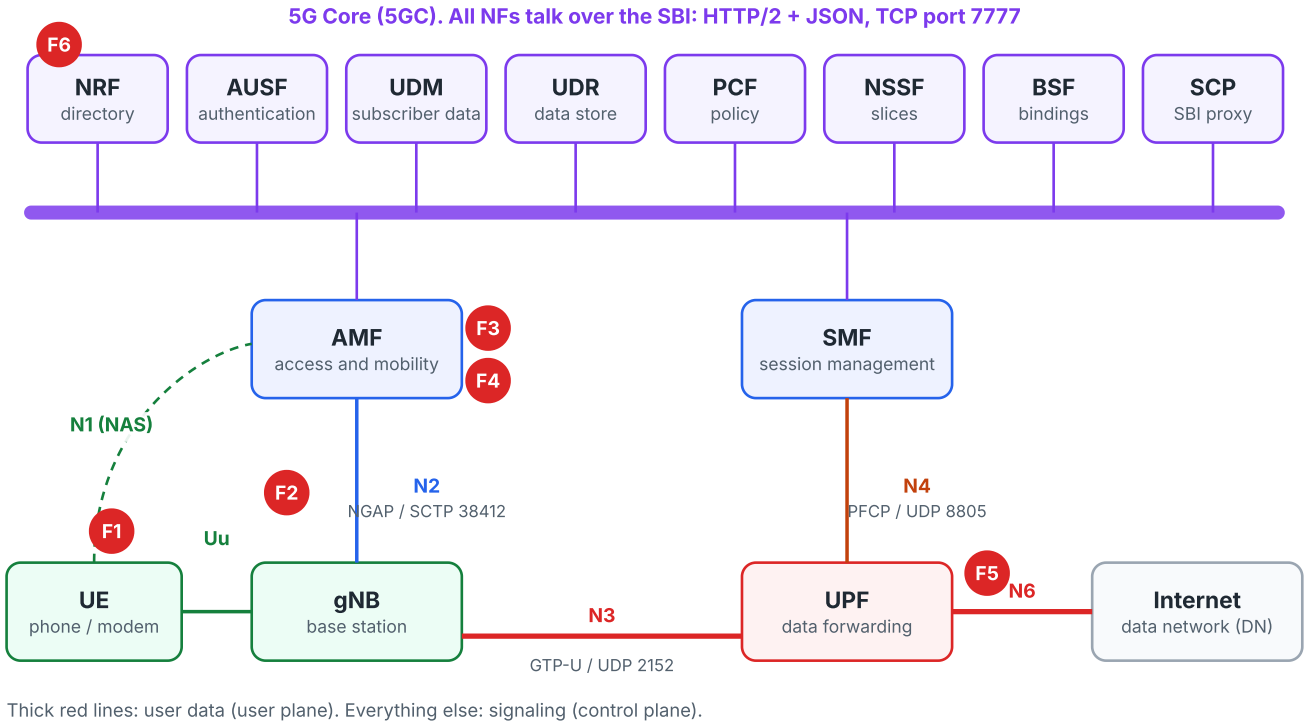



Figure 8. Where each fault was injected. F1 subscriber keys, F2 gNB setup, F3 slice, F4 DNN, F5 UPF, F6 NRF.

| # | What was broken | What the UE sees | Cause | Who rejected |
|---|---|---|---|---|
| F1 | UE OPc differs from the database by one digit | Authentication Reject | 5GMM #20 MAC failure | the UE rejected the network, then the AMF rejected the UE |
| F2 | gNB advertises a foreign PLMN 001/01 | no network at all | NGAP unknown-PLMN | the AMF refused the gNB |
| F3 | UE requests slice SST 2 | Registration Reject | 5GMM #62 | AMF |
| F4 | UE requests DNN "corp" | registered, no data | 5GMM #91 | AMF |
| F5 | UPF stopped | registered, session rejected | 5GSM #38 | SMF |
| F6 | NRF stopped | nothing, everything works | none | nobody (yet) |

## F1. Wrong OPc: "the keys do not match"

**Broken:** the last digit of the UE OPc changed from ...6183CA to ...6183CB. The database kept the old value.

**What happened:** registration reached Authentication Request (frame 158). The SIM checked AUTN with its (wrong) OPc, the MAC did not match, and the phone concluded that **the network is fake**: Authentication Failure, cause **#20 MAC failure** (frame 159). The AMF answered Authentication Reject (frame 160). The UE treats the SIM as invalid and stops trying.

**Telling them apart:** #20 MAC failure = K/OPc mismatch (database or SIM). #21 Synch failure = only the SQN drifted, the network resyncs on its own. #26 non-5G authentication unacceptable = wrong authentication type.

**In production:** one subscriber → check K/OPc in the database (often a SIM batch import error). Many subscribers from one SIM batch → wrong operator OP or algorithm.

## F2. Foreign PLMN on the gNB: "wrong cell site"

**Broken:** the gNB config set to PLMN 001/01 instead of 250/99.

**What happened:** at startup the gNB sends the AMF an **NGSetupRequest** with its PLMN (frame 5). The AMF serves only 250/99 and answers **NGSetupFailure**, cause *misc: unknown-PLMN-or-SNPN*, TimeToWait 10 s (frame 7). The phone never gets that far: there is no cell.




Figure 9. How the PLMN is encoded. Wireshark shows pLMNIdentity 00f110, which is 001/01.

**In production:** a new site does not come up → check NG Setup: PLMN, TAC and slices in the Supported TA List must match the AMF configuration.

## F3. Slice SST 2: "no such service"

**Broken:** the UE requests slice SST 2; the network only serves SST 1.

**What happened:** authentication passed (the keys are correct). The Requested NSSAI with SST 2 is only visible in Security Mode Complete (frame 111): the UE does not reveal it before security is on. The AMF answered **Registration Reject #62 No network slices available** (frame 112).

**In production:** compare the requested slices with three places: what the AMF supports in that tracking area, what the subscription allows (UDM), and what the NSSF permits. Watch the SD: "SST 1 + SD 000001" and "SST 1 without SD" are different slices.

## F4. DNN "corp": "not in the subscription"

**Broken:** the UE requests a session for DNN "corp"; the subscription only has "internet".

**What happened:** registration succeeded. During registration the AMF already fetched the subscribed DNNs (smf-select-data, frames 155–177). On the session request for "corp" (frame 243) the AMF **itself** answered with **5GMM #91** "DNN not supported or not subscribed in the slice" (frame 245) and never contacted an SMF.

**Key question** for any session failure: **who rejected?** A 5GMM cause (#91) from the AMF points to the subscription. A 5GSM cause (#27 missing or unknown DNN) from the SMF means the DNN is subscribed but the SMF/UPF do not know it.

A vendor detail: Open5GS answers with a 5GMM STATUS message. As I read the standard, a DL NAS Transport carrying the same #91 would be expected. Same cause, different message type.

## F5. UPF down: "the manager is in, the mover is not"

**Broken:** `systemctl stop open5gs-upfd`.

**What happened:**

- The SMF and UPF check each other over N4 about every 11 s (PFCP Heartbeat). After the stop, 3 requests went unanswered. The SMF declared the association lost and kept retrying PFCP Association Setup every few seconds.
- Registration worked: it does not need the UPF.
- On the session request the SMF started building it (context, PCF policy), found no UPF, rolled everything back and sent **PDU Session Establishment Reject, 5GSM #38 Network failure**.
- The reject carried no back-off timer, so the UE retried about every 1.1 s, 12 times. With thousands of phones that becomes a signaling storm, which is why a back-off timer in real rejects matters.
- After the UPF was started, the association came back on its own (22:02:12).

**In production:** subscribers register but sessions are rejected by the SMF → check N4 first: are heartbeats flowing between the SMF and every UPF?

## F6. NRF down: "the directory is gone, everyone keeps working"

**Broken:** `systemctl stop open5gs-nrfd` for about 65 seconds.

**What happened:** the UE registered and got a session (10.45.0.18) as if nothing was wrong. The NFs already knew each other from when the NRF was up. Every request through the SCP carries a `3gpp-sbi-target-apiroot` header with the target address, and the capture has no discovery request to the NRF at all. Only the NF heartbeats to the NRF suffered: the SCP got connection refused and returned HTTP 500 to the NFs, which retried registration every 2 s. When the NRF came back, they all re-registered on their own.

**What it means in production** (reasoning; 65 s in the lab did not get that far): an NRF outage is dangerous because it looks harmless at first. New NF instances cannot register, scale-out and failover stop working, and when the cached profiles expire everything fails at once. That is why production NRFs are always redundant.

## 12. Reading the captures in Wireshark

1. Open a file from `01-5gsa-core/pcap`.
2. **Analyze → Decode As**: SCTP port 38412 → NGAP; TCP port 7777 → HTTP2.
3. **Edit → Preferences → Protocols → NAS-5GS**: enable "Try to detect and decode 5G-EA0 ciphered messages". Without it, NAS after Security Mode is unreadable.
4. Useful filters:
   - `ngap`: everything between the gNB and the AMF;
   - `nas-5gs`: phone messages;
   - `pfcp`: SMF → UPF commands;
   - `gtp`: user data;
   - `http2.headers.path contains "nausf"`: requests to AUSF (swap for nudm, nsmf, npcf...);
   - `!(http2.headers.path contains "nnrf")`: hide background NRF heartbeats.
5. Remember the SCP: every SBI request appears twice (NF → 127.0.0.200, then 127.0.0.200 → target).

## 13. Key takeaways

- A 5G SA core (Open5GS + UERANSIM) with its own PLMN, analyzed frame by frame: registration with 5G-AKA, PDU session, GTP-U on N3, deregistration, and Service Request from CM-IDLE.
- Six failure scenarios: subscriber keys (#20), foreign PLMN on N2, slice (#62), DNN (#91 from the AMF vs #27 from the SMF), UPF loss (5GSM #38, PFCP heartbeat), NRF loss (cached profiles, risk to scaling).
- Triage order for "no data": is the subscriber registered; if yes, who rejected the session, AMF (5GMM) or SMF (5GSM); then N4 between SMF and UPF; then N3 and NAT/routing on N6.
- Single-host pitfalls solved: Docker FORWARD DROP, the UE/UPF routing loop (network namespace), rt_tables on Ubuntu 26.04.

> **Next:** Module 2, the LTE core (EPC) and Diameter.